Software Design Document (SDD)
Assignment #2 Disassembler for XE computer
CS530, Fall 2026

Team:
Alan Chavarin, cssc2513, Red ID: 827690364, [Design, Developer, Test]
Amir Ali, cssc2503, Red ID: 132395455, [Design, Developer, Documentation, Configuration Manager]

Overview & Goals:
Overview: We will build a SIC XE program named “lxe” that disassembles code from symbol and object files and returns a listing and sic file.
The primary goals of this project will entail the design, development, testing, and delivery of a working limited two-pass assembler for the XE version of the SIC/XE machine.

The assembler will open SIC XE assembly source files, process all XE features and assembler directives START, END, BYTE, WORD, RESB, RESW, BASE, and generate a .l listing file and a .st symbol table file for each input. No object code files will be generated.

Goals:

1. Open and validate one or more XE assembly source files (.sic) provided as command-line arguments.
2. Effectively perform Pass 1: scan each source line, assign location counter values, handle assembler directives, and populate the symbol table.
3. Effectively perform Pass 2: resolve symbol references, encode instructions for all formats, 1, 2, 3, and 4, with correct addressing modes, and build listing records.
4. Successfully write the formatted listing file (.l) matching the style of Figures 2.10 and 2.16 in the textbook, and the symbol table file (.st), for each assembled program.

Project Description:
The purpose of this project is to create a limited two-pass SIC XE assembler (lxe) that will open one or more given XE assembly source files.
The program will use an opcode table (OPTAB) to identify and encode each instruction's mnemonic, and a symbol table (SYMTAB) to resolve label references.
The output will produce a listing file and a symbol table file for each source file processed.

The assembler supports all the following XE-mode features:

- Format 1, 2, 3, and 4 instructions
- immediate ('#'), indirect ('@'), and indexed (',X') addressing; the '+' prefix for Format 4;
- PC-relative and base-relative addressing (BASE/NOBASE directives);
- Directives START, END, BYTE, WORD, RESB, and RESW.

There will be error handling if input files are not found or can’t be opened.

Plan of Action and Milestones: Mar 10, 2026 to Apr 21, 2026 (5 Weeks)

First Phase: Week 1 (March 10 to March 17)

- Brainstorm timeframe, team roles and workload distribution.
- Workload Distribution between team members.

Phase 2: Week 2 (March 18 to March 25)

- Create a Data Flow Diagram and module breakdown.
- Sketch some pseudocode for Pass 1 and Pass 2.
- Assign who will code which aspect of the project.

Phase 3: Week 3 (March 26 to April 2)

- Code each module from pseudocode
- Combine modules and test integrations
- Debug and stamp out any errors found in integration

Phase 4: 4 (April 3 to April 10)

- Finalize all coding requirements
- Combine modules and test integrations
- Debug and stamp out any errors found in integration

Phase 5: Week 5 (April 11 to April 21)

- Make a final executable
- Finalize and organize all documentation, such as README, comments, SDD, and Makefile
- Test to ensure flawless functionality.

Functional and System Requirements:

- The program will be named lxe and accept one or more XE assembly source file names as command-line arguments.
- If no arguments are provided, lxe will print an error message and terminate.
- lxe will open and parse each source file, scanning every assembler directive and instruction using the two-pass assembler algorithm.
- Pass 1 will initialize and maintain a location counter (LOCCTR) from the START address, compute sizes for all instructions and directives, and populate the SYMTAB with (symbol, address) pairs.
- Pass 2 will encode Format 1, 2, 3, and 4 instructions with correct opcode bytes, register fields, and addressing mode flag bits (n, i, x, b, p, e).
- Pass 2 will resolve symbol references through SYMTAB lookup and flag undefined symbols as errors.
- Pass 2 will handle PC-relative and base-relative addressing for Format 3; BASE and NOBASE directives will control the base register.
- For each input file foo.sic, lxe will produce foo.l (listing file) and foo.st (symbol table file).
- The listing file format will match Figures 2.10 and 2.16 of the textbook.
- Directives implemented: START, END, BYTE, WORD, RESB, RESW, BASE, NOBASE. EQU and ORG are **not** implemented.
- Edoras handles the compiler (GCC), OS, system hardware requirements.

System Design/Specification:

Parser (parser.h / parser.cpp)

Tokenizes each source line into a SourceLine struct with fields: label, opcode, operand, comment, and line_number. Handles the '+' Format 4 prefix on opcodes and '#', '@', ',X' modifiers on operands.

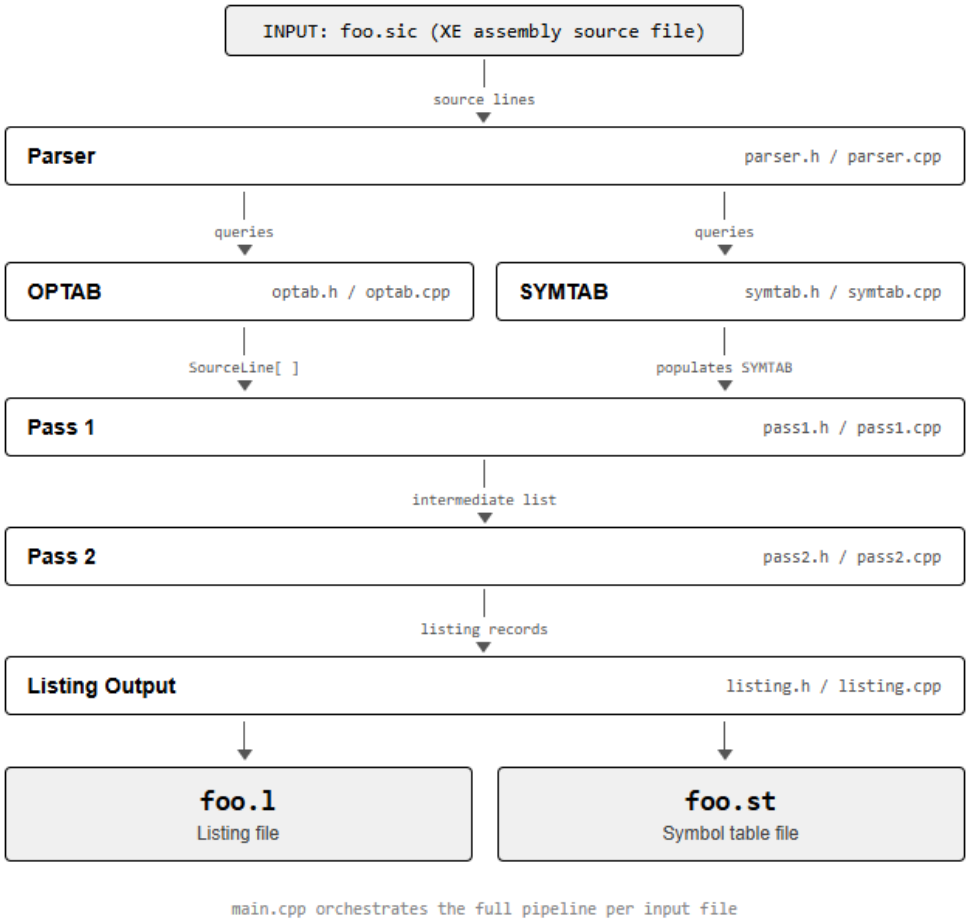
OPTAB (optab.h / optab.cpp)
A static lookup table mapping each SIC/XE mnemonic to its opcode byte and valid instruction formats. Queried during both passes. Hardcoded to cover all opcodes START, END, BYTE, WORD, RESB, RESW, BASE.

SYMTAB (symtab.h / symtab.cpp)
Maps symbol names (strings) to their integer addresses. Provides insert(name, addr), lookup(name), and dump(file) for .st output. Duplicate label definitions are detected and flagged here.

Pass 1 (pass1.h / pass1.cpp)
It iterates parsed source lines, manages LOCCTR, handles all directives, computes instruction sizes for all formats, and populates the SYMTAB. Produces an annotated intermediate list consumed by Pass 2.

Pass 2 (pass2.h / pass2.cpp)
Re-iterates the intermediate list. For each instruction, selects the correct format and addressing mode (PC-relative first, then base-relative), encodes the object code, and fills listing records. Tracks the BASE register for base-relative addressing.

Diagram:



Listing Output (listing.h / listing.cpp)
Accumulates listing records (address, object code hex, original source line) and formats them into the .l output file matching the textbook figure style.

Development Environment:
List all systems, tools, APIs, compilers, editors, etc used for the design and development of your project.

OS: Linux, edoras.sdsu.edu (primary build and test target)

Compiler: GCC / G++ (Edoras).

Build System: make with a Makefile targeting the Edoras environment, and results in an executable named lxe.
Text Editor: VIM

Run/Test Environment:
Platform: edoras.sdsu.edu (Linux).

Executable: lxe, produced by make in ~/a2/.

Invocation: ./lxe file1.sic [file2.sic ...]

Output files (.l and .st) written to the same directory as the input files.

Using the sample files provided on Canvas, we will first run and test on a local GCC compiler, then compile and test on Edoras to ensure there are no platform-specific errors. Additional test cases will be written to cover edge cases and ensure the program runs correctly under all expected conditions.

Code and Test Plan:

1. Code Review

- Both team members will review each other's source code.
- Check that all coding conventions are followed and all project requirements are met.

2. Unit Testing

- Each module (parser, OPTAB, SYMTAB, Pass 1, Pass 2, listing output) will be tested individually to ensure it works as intended before integration.
- Each module meets the requirements stated in the project specification.

3. Integration Testing

- Both members will combine all modules and test them together as a complete program.
- Primary integration test cases are the textbook sample programs (equivalent to Figures 2.9 and 2.15); expected output matches Figures 2.10 and 2.16.
- Additional test cases will cover: no command-line arguments, missing input files, Format 4 instructions, multi-file invocation, and assembly errors (undefined symbols, duplicate labels).
- A diff tool will be used to compare generated .l output against expected listing files.

Reference & Examples: System Software: An Introduction to Systems Programming, Leland Beck — Sections 2.1–2.3.4.

Figure 2.9 (Page 83)

Line	Source statement			
5	COPY	START	0	COPY FILE FROM INPUT TO OUTPUT
10	FIRST	STL	RETADR	SAVE RETURN ADDRESS
13		LDB	#LENGTH	ESTABLISH BASE REGISTER
14		BASE	LENGTH	
15	CLOOP	+JSUB	RDREC	READ INPUT RECORD
20		LDA	LENGTH	TEST FOR EOF (LENGTH = 0)
25		COMP	#0	
30		JEQ	ENDFIL	EXIT IF EOF FOUND
35		+JSUB	WRREC	WRITE OUTPUT RECORD
40		J	CLOOP	LOOP
45	ENDFIL	LDA	=C'EOF'	INSERT END OF FILE MARKER
50		STA	BUFFER	
55		LDA	#3	SET LENGTH = 3
60		STA	LENGTH	
65		+JSUB	WRREC	WRITE EOF
70		J	@RETADR	RETURN TO CALLER
93		LTORG		
95	RETADR	RESW	1	
100	LENGTH	RESW	1	LENGTH OF RECORD
105	BUFFER	RESB	4096	4096-BYTE BUFFER AREA
106	BUFEND	EQU	*	
107	MAXLEN	EQU	BUFEND-BUFFER	MAXIMUM RECORD LENGTH
110	.			
115	.	SUBROUTINE TO READ RECORD INTO BUFFER		
120	.			
125	RDREC	CLEAR	X	CLEAR LOOP COUNTER
130		CLEAR	A	CLEAR A TO ZERO
132		CLEAR	S	CLEAR S TO ZERO
133		+LDT	#MAXLEN	
135	RLOOP	TD	INPUT	TEST INPUT DEVICE
140		JEQ	RLOOP	LOOP UNTIL READY
145		RD	INPUT	READ CHARACTER INTO REGISTER A
150		COMPR	A,S	TEST FOR END OF RECORD (X'00')
155		JEQ	EXIT	EXIT LOOP IF EOR
160		STCH	BUFFER,X	STORE CHARACTER IN BUFFER
165		TIXR	T	LOOP UNLESS MAX LENGTH
170		JLT	RLOOP	HAS BEEN REACHED
175	EXIT	STX	LENGTH	SAVE RECORD LENGTH
180		RSUB		RETURN TO CALLER
185	INPUT	BYTE	X'F1'	CODE FOR INPUT DEVICE
195	.			
200	.	SUBROUTINE TO WRITE RECORD FROM BUFFER		
205	.			
210	WRREC	CLEAR	X	CLEAR LOOP COUNTER
212		LDT	LENGTH	
215	WLOOP	TD	=X'05'	TEST OUTPUT DEVICE
220		JEQ	WLOOP	LOOP UNTIL READY
225		LDCH	BUFFER,X	GET CHARACTER FROM BUFFER
230		WD	=X'05'	WRITE CHARACTER
235		TIXR	T	LOOP UNTIL ALL CHARACTERS
240		JLT	WLOOP	HAVE BEEN WRITTEN
245		RSUB		RETURN TO CALLER
255		END	FIRST	

Figure 2.9 Program demonstrating additional assembler features.

Figure 2.10 (Page 84)

Line	Loc	Source statement			Object code
5	0000	COPY	START	0	
10	0000	FIRST	STL	RETADR	17202D
13	0003		LDB	#LENGTH	69202D
14			BASE	LENGTH	
15	0006	CLOOP	+JSUB	RDREC	4B101036
20	000A		LDA	LENGTH	032026
25	000D		COMP	#0	290000
30	0010		JEQ	ENDFIL	332007
35	0013		+JSUB	WRREC	4B10105D
40	0017		J	CLOOP	3F2FEC
45	001A	ENDFIL	LDA	=C'EOF'	032010
50	001D		STA	BUFFER	0F2016
55	0020		LDA	#3	010003
60	0023		STA	LENGTH	0F200D
65	0026		+JSUB	WRREC	4B10105D
70	002A		J	@RETADR	3E2003
93			LTORG		
	002D	*	=C'EOF'		454F46
95	0030	RETADR	RESW	1	
100	0033	LENGTH	RESW	1	
105	0036	BUFFER	RESB	4096	
106	1036	BUFEND	EQU	*	
107	1000	MAXLEN	EQU	BUFEND-BUFFER	
110		.			
115		.	SUBROUTINE TO READ RECORD INTO BUFFER		
120		.			
125	1036	RDREC	CLEAR	X	B410
130	1038		CLEAR	A	B400
132	103A		CLEAR	S	B440
133	103C		+LDT	#MAXLEN	75101000
135	1040	RLOOP	TD	INPUT	E32019
140	1043		JEQ	RLOOP	332FFA
145	1046		RD	INPUT	DB2013
150	1049		COMPR	A,S	A004
155	104B		JEQ	EXIT	332008
160	104E		STCH	BUFFER,X	57C003
165	1051		TIXR	T	B850
170	1053		JLT	RLOOP	3B2FEA
175	1056	EXIT	STX	LENGTH	134000
180	1059		RSUB		4F0000
185	105C	INPUT	BYTE	X'F1'	F1
195		.			
200		.	SUBROUTINE TO WRITE RECORD FROM BUFFER		
205		.			
210	105D	WRREC	CLEAR	X	B410
212	105F		LDT	LENGTH	774000
215	1062	WLOOP	TD	=X'05'	E32011
220	1065		JEQ	WLOOP	332FFA
225	1068		LDCH	BUFFER,X	53C003
230	106B		WD	=X'05'	DF2008
235	106E		TIXR .	T	B850
240	1070		JLT	WLOOP	3B2FEF
245	1073		RSUB		4F0000
255			END	FIRST	
	1076	*	=X'05'		05

Figure 2.10 Program from Fig. 2.9 with object code.

Figure 2.15 (Page 102)

```
Line        Source statement

  5    COPY       START      0                 COPY FILE FROM INPUT TO OUTPUT
  6               EXTDEF     BUFFER,BUFEND,LENGTH
  7               EXTREF     RDREC,WRREC
 10    FIRST      STL        RETADR            SAVE RETURN ADDRESS
 15    CLOOP     +JSUB       RDREC             READ INPUT RECORD
 20               LDA        LENGTH            TEST FOR EOF (LENGTH = 0)
 25               COMP      #0
 30               JEQ        ENDFIL            EXIT IF EOF FOUND
 35              +JSUB       WRREC             WRITE OUTPUT RECORD
 40               J          CLOOP             LOOP
 45    ENDFIL     LDA       =C'EOF'            INSERT END OF FILE MARKER
 50               STA        BUFFER
 55               LDA       #3                 SET LENGTH = 3
 60               STA        LENGTH
 65              +JSUB       WRREC             WRITE EOF
 70               J         @RETADR            RETURN TO CALLER
 95    RETADR     RESW       1
100    LENGTH     RESW       1                 LENGTH OF RECORD
103               LTORG
105    BUFFER     RESB       4096              4096-BYTE BUFFER AREA
106    BUFEND     EQU        *
107    MAXLEN     EQU        BUFEND-BUFFER

109    RDREC      CSECT
110    .
115    .          SUBROUTINE TO READ RECORD INTO BUFFER
120    .
122               EXTREF     BUFFER,LENGTH,BUFEND
125               CLEAR      X                 CLEAR LOOP COUNTER
130               CLEAR      A                 CLEAR A TO ZERO
132               CLEAR      S                 CLEAR S TO ZERO
133               LDT        MAXLEN
135    RLOOP      TD         INPUT             TEST INPUT DEVICE
140               JEQ        RLOOP             LOOP UNTIL READY
145               RD         INPUT             READ CHARACTER INTO REGISTER A
150               COMPR      A,S               TEST FOR END OF RECORD (X'00')
155               JEQ        EXIT              EXIT LOOP IF EOR
160              +STCH       BUFFER,X          STORE CHARACTER IN BUFFER
165               TIXR       T                 LOOP UNLESS MAX LENGTH
170               JLT        RLOOP               HAS BEEN REACHED
175    EXIT      +STX        LENGTH            SAVE RECORD LENGTH
180               RSUB                         RETURN TO CALLER
185    INPUT      BYTE       X'F1'             CODE FOR INPUT DEVICE
190    MAXLEN     WORD       BUFEND-BUFFER

193    WRREC      CSECT
195    .
200    .          SUBROUTINE TO WRITE RECORD FROM BUFFER
205    .
207               EXTREF     LENGTH,BUFFER
210               CLEAR      X                 CLEAR LOOP COUNTER
212              +LDT        LENGTH
215    WLOOP      TD        =X'05'             TEST OUTPUT DEVICE
220               JEQ        WLOOP             LOOP UNTIL READY
225              +LDCH       BUFFER,X          GET CHARACTER FROM BUFFER
230               WD        =X'05'             WRITE CHARACTER
235               TIXR       T                 LOOP UNTIL ALL CHARACTERS
240               JLT        WLOOP               HAVE BEEN WRITTEN
245               RSUB                         RETURN TO CALLER
255               END        FIRST
```

Figure 2.15 Illustration of control sections and program linking.

Figure 2.16 (Page 103)

```
Line   Loc    Source statement                    Object code
  5    0000   COPY     START    0
  6                    EXTDEF   BUFFER,BUFEND,LENGTH
  7                    EXTREF   RDREC,WRREC
 10    0000   FIRST    STL      RETADR            172027
 15    0003   CLOOP   +JSUB     RDREC             4B100000
 20    0007            LDA      LENGTH            032023
 25    000A            COMP    #0                 290000
 30    000D            JEQ      ENDFIL            332007
 35    0010           +JSUB     WRREC             4B100000
 40    0014            J        CLOOP             3F2FEC
 45    0017   ENDFIL   LDA     =C'EOF'            032016
 50    001A            STA      BUFFER            0F2016
 55    001D            LDA     #3                 010003
 60    0020            STA      LENGTH            0F200A
 65    0023           +JSUB     WRREC             4B100000
 70    0027            J       @RETADR            3E2000
 95    002A   RETADR   RESW     1
100    002D   LENGTH   RESW     1
103                    LTORG
       0030   *       =C'EOF'                     454F46
105    0033   BUFFER   RESB     4096
106    1033   BUFEND   EQU      *
107    1000   MAXLEN   EQU      BUFEND-BUFFER

109    0000   RDREC    CSECT
110              .
115              .     SUBROUTINE TO READ RECORD INTO BUFFER
120              .
122                    EXTREF   BUFFER,LENGTH,BUFEND
125    0000            CLEAR    X                 B410
130    0002            CLEAR    A                 B400
132    0004            CLEAR    S                 B440
133    0006            LDT      MAXLEN            77201F
135    0009   RLOOP    TD       INPUT             E3201B
140    000C            JEQ      RLOOP             332FFA
145    000F            RD       INPUT             DB2015
150    0012            COMPR    A,S               A004
155    0014            JEQ      EXIT              332009
160    0017           +STCH     BUFFER,X          57900000
165    001B            TIXR     T                 B850
170    001D            JLT      RLOOP             3B2FE9
175    0020   EXIT    +STX      LENGTH            13100000
180    0024            RSUB                       4F0000
185    0027   INPUT    BYTE     X'F1'             F1
190    0028   MAXLEN   WORD     BUFEND-BUFFER     000000

193    0000   WRREC    CSECT
195              .
200              .     SUBROUTINE TO WRITE RECORD FROM BUFFER
205              .
207                    EXTREF   LENGTH,BUFFER
210    0000            CLEAR    X                 B410
212    0002           +LDT      LENGTH            77100000
215    0006   WLOOP    TD      =X'05'             E32012
220    0009            JEQ      WLOOP             332FFA
225    000C           +LDCH     BUFFER,X          53900000
230    0010            WD      =X'05'             DF2008
235    0013            TIXR     T                 B850
240    0015            JLT      WLOOP             3B2FEE
245    0018            RSUB                       4F0000
255                    END      FIRST
       001B   *       =X'05'                      05
```

Figure 2.16 Program from Fig. 2.15 with object code.